Mémo Node.js & NPM - DWWM

📦 Commandes NPM

- `npm init -y` : Initialiser un projet (crée package.json)
- `npm install ` : Installer un package (ex: express)
- `npm install --save-dev nodemon` : Installer pour le développement
- `npm install` : Réinstaller tous les modules (basé sur package.json)

🌐 Codes HTTP Courants

- **200 OK** : Succès de la requête
- **201 Created** : Succès et création de ressource (POST)
- **400 Bad Request** : Erreur dans les données envoyées
- **401 Unauthorized** : Authentification nécessaire
- **404 Not Found** : La ressource n'existe pas
- **500 Internal Server Error** : Crash du serveur

🚀 Node.js Express Snippet

- `const req.params` : Récupérer des variables d'URL (/user/:id)
- `const req.body` : Récupérer des données de formulaire ou JSON
- `res.json()` : Envoyer des données au format JSON
- `res.status()` : Définir le code de réponse HTTP

Mémo Node.js & NPM - DWWM

L’essentiel pour démarrer un projet sous Node.js :

Voici une fiche de synthèse structurée pour aider à démarrer votre premier serveur.

1. Initialiser le projet

Avant d'écrire la moindre ligne de code, il faut préparer l'environnement Node.js.

- **`npm init -y`** : Cette commande crée un fichier `package.json` avec les réglages par défaut. C’est la "carte d’identité" de votre projet (nom, version, dépendances).
- **`npm install express`** : Installe le framework Express (le standard pour créer des serveurs facilement).

2. Structure de base d'un serveur (app.js)

Voici le code minimaliste pour lancer un serveur. **L'ordre des lignes est crucial.**

JavaScript

```
// 1. Importation du module Express
const express = require('express');

// 2. Création de l'instance de l'application
const app = express();

// 3. Définition du PORT
// On utilise souvent 3000 par convention en développement
const PORT = 3000;

// 4. Définition d'une "Route" (le point d'entrée)
// Quand l'utilisateur arrive sur la racine "/"
app.get('/', (req, res) => {
    res.send('Bienvenue sur notre premier serveur Node !');
});
```

```
// 5. Lancement du serveur
app.listen(PORT, () => {
    console.log(`Serveur démarré sur : http://localhost:$
{PORT}`);
});
```

3. Les notions clés à retenir

Le cycle Requête / Réponse

Tout le web repose sur ce mécanisme. Vous devez manipuler deux objets principaux dans vos fonctions :

- **`req` (Request)** : Ce que l'utilisateur envoie (paramètres, données de formulaire).
- **`res` (Response)** : Ce que le serveur renvoie (HTML, JSON, message d'erreur).

Le fichier `.gitignore`

Indispensable dès le début ! Rappelez-leur de créer un fichier `.gitignore` et d'y ajouter `node_modules`. On ne partage jamais ce dossier (trop lourd et généré automatiquement via `npm install`).
Le fichier `.gitignore` est un simple fichier texte situé à la racine de ton projet. Son rôle est de dire à **Git** : "S'il te plaît, ignore ces fichiers et ces dossiers, ne les surveille pas et ne les envoie jamais sur le dépôt distant (GitHub, GitLab, etc.)."

Nodemon (Le confort)

Pour éviter de couper et relancer le serveur à chaque modification (`node app.js`), conseillez-il est préférable d'utiliser **Nodemon** :

- Installation : `npm install --save-dev nodemon`
- Usage : `npx nodemon app.js`

4. Les fonctions essentielles (Middleware & Routing)

Pour aller un peu plus loin, voici comment gérer des données simples :

JavaScript

```
// Middleware pour permettre à Express de lire le JSON
app.use(express.json());
```

Mémo Node.js & NPM - DWWM

```
// Exemple de route avec un paramètre dynamique
app.get('/user/:name', (req, res) => {
    const userName = req.params.name;
    res.send(`Profil de l'utilisateur : ${userName}`);
});
```

1. Exemple de code (Route GET)

```
// On liste les ID des tâches
app.get('/tasks/ids', (req, res) => {
   // .map() parcourt le tableau 'tasks' et ne garde que la propriété 'id'
   const allIds = tasks.map(task => task.id);

   // On renvoie le résultat au format JSON
   res.json(allIds);
});
```

2. Explication simplifiée

Pour bien comprendre, vous pouvez décomposer l'action ainsi :

- **Le "Nettoyage" :** Le serveur possède un tableau d'objets complets (avec titre, statut, date, etc.). Mais ici, on ne veut que les "étiquettes" (les IDs).
- **La transformation :** On utilise souvent la méthode JavaScript `.map()`. C'est comme un filtre qui ne laisserait passer que la colonne "ID" d'un tableau Excel.
- **L'utilité :** C'est très fréquent en développement pour :
  - Vérifier si un ID existe avant de faire une modification.
  - Créer des menus de navigation dynamiques.
  - Optimiser la bande passante (on n'envoie que le strict nécessaire au lieu de 500 objets complets).

Mémo Node.js & NPM - DWWM

```
if (task) {
    res.json(task);
} else {
  res.status(404).json({ error: 'Tâche non trouvée' });
}
});
```

1. Traduction "en français" du code

"Si la tâche a été trouvée (si `task` n'est pas vide), alors renvoie-la en JSON. Sinon, change le code de réponse à **404** et renvoie un message d'erreur."

2. Pourquoi c'est important ? (Le statut 404)

C'est le point clé ici. Par défaut, Express renvoie toujours un code **200 OK**.

- Si vous ne mettez pas le `else`, et que la tâche n'existe pas, le serveur risque de renvoyer `null` ou de planter, tout en disant "200 OK" au navigateur.
- En mettant `res.status(404)`, on communique "proprement" avec le client (le navigateur ou l'application mobile) pour lui dire explicitement : **"J'ai bien reçu ta demande, mais l'objet n'existe pas dans ma base."**

Analogie pour comprendre

C'est comme un serveur au restaurant :

1. Vous demandez un plat (la requête).
2. Le serveur va en cuisine (le code cherche dans le tableau).
3. **S'il y en a :** Il vous l'apporte avec un sourire (Code 200 + Données).
4. **S'il n'y en a plus :** Il revient vous dire qu'il est en rupture de stock (Code 404 + Message d'erreur).